

Computational Geometry 2nd assignment

Zaza Maria

March 2026

1 Geometric Predicates and Numerical Robustness in CGAL

Geometric algorithms rely on **predicates**: functions that return a discrete answer such as `LEFT_TURN`, `RIGHT_TURN`, or `COLLINEAR`. The correctness of these predicates depends critically on the arithmetic used to evaluate them.

The *orientation predicate* for three points $p = (p_x, p_y)$, $q = (q_x, q_y)$, $r = (r_x, r_y)$ is defined by the sign of the 2×2 determinant:

$$\Delta(p, q, r) = \begin{vmatrix} q_x - p_x & r_x - p_x \\ q_y - p_y & r_y - p_y \end{vmatrix} = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x).$$
$$\text{orientation}(p, q, r) = \begin{cases} \text{LEFT_TURN} & \Delta > 0, \\ \text{RIGHT_TURN} & \Delta < 0, \\ \text{COLLINEAR} & \Delta = 0. \end{cases}$$

In this assignment you will implement and compare several strategies for evaluating this predicate and observe how IEEE 754 floating-point errors can silently corrupt geometric computations.

Exercise 1. *Implement a function "user_orientation" that computes the orientation of three points using the standard 2×2 determinant in `double` arithmetic.*

Report

In Exercise 1, I implemented a function called `user_orientation` that computes the orientation of three points in the plane using the standard 2×2 determinant in `double` arithmetic.

The function takes the coordinates of three points p , q , and r , and computes

$$\text{det} = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x).$$

The sign of this determinant determines the result:

- if `det` > 0 , the result is `LEFT_TURN`,

- if `det < 0`, the result is `RIGHT_TURN`,
- if `det = 0`, the result is `COLLINEAR`.

I also defined an enum `Orientation` to represent these three possible cases, and a helper function `orientation_to_string` to print the result in a readable way.

Exercise 2. Consider the following three points:

$$p = (0, 0), \quad q = (2^{-537}, 2^{-537}), \quad r = (0, 2^{-538}).$$

The exact determinant is:

$$\Delta = 2^{-537} \cdot 2^{-538} - 2^{-537} \cdot 0 = 2^{-1075} > 0 \implies \text{LEFT_TURN}.$$

Use `std::ldexp` to construct the coordinates:

```
double px = 0.0,           py = 0.0;
double qx = std::ldexp(1.0, -537), qy = std::ldexp(1.0, -537);
double rx = 0.0,           ry = std::ldexp(1.0, -538);
```

For each CGAL kernel: *Simple cartesian*, *Exact-predicates-inexact-constructions-kernel (EPICK)* and *Exact-predicates-exact-constructions-kernel (EPECK)* and *"user_orientation"* function, call and print the orientation result.

Report

In Exercise 2, I tested the orientation of the three points

$$p = (0, 0), \quad q = (2^{-537}, 2^{-537}), \quad r = (0, 2^{-538}).$$

I used `std::ldexp` to construct the coordinates and compared the result of my `user_orientation` function with the CGAL kernels `Simple_cartesian<double>`, `EPICK`, and `EPECK`.

The implementations based on plain `double` arithmetic returned `COLLINEAR`, because the determinant is extremely small and underflows to zero. In contrast, `EPICK` and `EPECK` returned `LEFT_TURN`, because they use exact predicates.

Exercise 3. Implement *"user_orientation_robust"* function a (more) robust version of *"user_orientation"* function that computes the correct result.

Report

In Exercise 3, I implemented a more robust version of the orientation function, called `user_orientation_robust`.

The original `user_orientation` computes the determinant using `double` arithmetic. This can fail in numerically delicate cases. In the example from Exercise 2, the exact determinant is 2^{-1075} , which is positive, but it is too small

to be represented as a `double`. Therefore, the computed determinant underflows to zero and the function incorrectly returns `COLLINEAR`.

To avoid this problem, the robust version uses the Boost Multiprecision type `boost::multiprecision::cpp_dec_float_100`, included from `<boost/multiprecision/cpp_dec_float.hpp>`. This type provides about 100 decimal digits of precision, compared to about 15 decimal digits for `double` on my system. As a result, the determinant does not underflow to zero, and the function returns the correct result.

Therefore, for the difficult input of Exercise 2, `user_orientation` returns `COLLINEAR`, while `user_orientation_robust` correctly returns `LEFT_TURN`.

Exercise 4. *Fill in the table below and explain **why** each implementation gives that result:*

<i>Implementation</i>	<i>Result</i>	<i>Correct?</i>	<i>Reason</i>
<code>user_orientation</code>			
<code>user_orientation_robust</code>			
<code>Simple_cartesian<double></code>			
<code>EPICK</code>			
<code>EPECK</code>			

Report

<i>Implementation</i>	<i>Result</i>	<i>Correct?</i>	<i>Reason</i>
user_orientation	COLLINEAR	No	It computes the determinant in <code>double</code> . The exact value is 2^{-1075} , which is too small for <code>double</code> and underflows to 0. Therefore, the function returns <code>COLLINEAR</code> .
user_orientation_robust	LEFT_TURN	Yes	It uses the Boost <code>Multiprecision</code> type, which has more precision and a larger range than <code>double</code> . In this case, the determinant keeps the correct positive sign, so the function returns <code>LEFT_TURN</code> .
Simple_cartesian<double>	COLLINEAR	No	This kernel uses <code>double</code> coordinates and standard floating-point arithmetic. So it has the same numerical problem as <code>user_orientation</code> : the determinant underflows to zero.
EPICK	LEFT_TURN	Yes	EPICK uses exact predicates. Even if the coordinates are stored in inexact form, the orientation test is evaluated robustly, so the correct sign is returned.
EPECK	LEFT_TURN	Yes	EPECK uses exact predicates and exact constructions. Therefore, the orientation is computed exactly and the result is <code>LEFT_TURN</code> .

Exercise 5 (bonus). *Create three new points such that your new function “user_orientation_robust” computes the wrong result. What does EPICK kernel return for that input? Explain the results.*

Report The three points I created:

$$p = (10^{20}, 10^{20}), \quad q = (10^{20} + 1, 10^{20}), \quad r = (10^{20}, 10^{20} + 1).$$

Mathematically, these points form a left turn. However, when the coordinates are stored as `double`, the values $10^{20} + 1$ are rounded to 10^{20} , because `double` does not have enough precision at that magnitude.

Therefore, the three points become indistinguishable from

$$p = q = r = (10^{20}, 10^{20})$$

at the input level.

As a result, `user_orientation_robust` returns `COLLINEAR`, even though the intended mathematical points form a `LEFT_TURN`.

EPICK also returns `COLLINEAR` for the same `double` input, because the lost information cannot be recovered after the coordinates have already been rounded. (EPICK also uses the boost multi precision type when the predicate is close to zero)

Exercise 6 (Segment intersection). *Consider two segments with **integer-only** endpoints:*

$$seg_1 : A = (0, 0) \rightarrow B = (3, 1), \quad seg_2 : C = (1, 0) \rightarrow D = (1, 3).$$

- (a) *Compute the intersection point exactly by hand. Why is its y-coordinate not exactly representable in binary floating point?*
- (b) *Compute the intersection point in CGAL using both EPICK and EPECK. Explain the results.*

Solution

(a) Intersection point computed by hand.

$$seg_1 : y = \frac{1}{3}x, \quad seg_2 : x = 1$$

Hence their intersection is: $(1, \frac{1}{3})$.

The y -coordinate is not exactly representable in binary floating point, since $\frac{1}{3}$ is not a fraction that can be represented in a power of 2.

As a result a floating point type such as `double` can only store an approximation

of its value, not the exact number.

(b) Intersection in CGAL with EPICK and EPECK.

I computed the intersection point of two 2D segments using CGAL and compared the behavior of two different kernels:

EPICK (`Exact_predicates_inexact_constructions_kernel`) and
EPECK (`Exact_predicates_exact_constructions_kernel`).

The segments intersect at the point $(1, \frac{1}{3})$.

With EPICK, the result is printed in floating-point form, so the y -coordinate appears as an approximation (0.3333333333333337).

With EPECK, With constructions are exact, so the intersection coordinate is stored exactly as the rational number $1/3$. When the point is printed directly, CGAL still displays a decimal approximation, but using `CGAL::exact` reveals the exact stored value.

This shows the main difference between the two kernels: both detect the correct intersection, but EPECK preserves exact geometric constructions, while EPICK may introduce small numerical errors in the coordinates.

2 Convex Hull Algorithms and Experimental Complexity

Given a set P of n points in the plane, the **convex hull** $CH(P)$ is the smallest convex polygon containing all points. The three algorithms consider in this exercise are:

Algorithm	CGAL function	Complexity
Graham–Andrew scan	<code>ch_graham_andrew</code>	$O(n \log n)$
Jarvis March	<code>ch_jarvis</code>	$O(n \cdot h)$
Akl–Toussaint	<code>ch_aki_toussaint</code>	$O(n \log n)$

Exercise 7.

(a) Generate points using three random point generators, that produce:

- (i) n points uniformly distributed **on a circle** of radius R ,*
- (ii) n points uniformly distributed **inside a disk** of radius R*
- (iii) n points uniformly distributed **inside a square** with edge R .*

Benchmark Graham–Andrew scan vs. Jarvis March on all distributions for $n \in \{1000, 5000, 10000, 50000\}$ and $R = 1$. Collect results in a table or graph and explain the results.

(b) Generate a pointset where Jarvis March is faster than Graham–Andrew and explain.

(c) Add *ch_aki_toussaint* to your comparisons from questions (a) and (b). Explain the results.

(d) Experiment by doing the above computations with both *Simple_cartesian* and *EPICK* kernels and explain the results.

Report

Sub-questions (c) and (d), were implemented in parallel with (a) and (b).

(a)

Distribution	Points	Hull size	Graham-Andrew (ms)	Jarvis (ms)	Akl-Toussaint (ms)
Circle	1000	1000	0.0609	0.5893	0.0479
Circle	5000	5000	0.2498	10.3976	0.2660
Circle	10000	10000	0.5489	43.0985	0.5730
Circle	50000	50000	3.2485	1088.6140	3.1852
Disk	1000	34	0.0179	0.0250	0.0207
Disk	5000	52	0.1670	0.1423	0.1065
Disk	10000	71	0.4402	0.3496	0.2441
Disk	50000	131	2.4683	2.9942	1.3363
Square	1000	17	0.0371	0.0135	0.0291
Square	5000	18	0.1640	0.0548	0.1294
Square	10000	25	0.3897	0.1395	0.2692
Square	50000	30	2.5364	0.8311	1.9572

Table 1: Benchmark results for the three convex hull algorithms using the Simple Cartesian kernel.

(a) When more points are generated on the hull Jarvis becomes clearly slower than Graham-Andrews, since Jarvis is output sensitive and his implementation time grows larger as the points on the hull increase.

(c) Akl-Toussain has the same worst case time as Graham-Andrew, but in cases where there are much more internal points than the number of points on the hull, Akl-Toussaint is faster than Graham since this algorithm follows a method that finds all the extreme points of the generated set and throws away their internal points, since these points will certainly not be part of the hull, thus the convex hull computation is left significantly less points to examine, which results to a better running time, since

$$O(n) + O(m \log m)$$

, where m is the number of points that remain after filtering.

For the worst case (circle) no points are thrown away by Akl-Toussaint and the convex hull computation takes $O(n \log n)$ time, same as Graham-Andrew.

Distribution	Points	Hull size	Graham–Andrew (ms)	Jarvis (ms)	Akl–Toussaint (ms)
Circle	1000	1000	0.1256	54.6432	0.1065
Circle	5000	5000	0.3881	776.0191	0.2749
Circle	10000	10000	0.5850	3119.8326	0.5979
Circle	50000	50000	3.4375	81826.7766	3.4016
Disk	1000	34	0.0411	1.1291	0.0290
Disk	5000	52	0.2254	9.0530	0.1490
Disk	10000	71	0.4825	22.5144	0.2768
Disk	50000	131	2.7439	207.8242	1.5922
Square	1000	17	0.0375	0.5757	0.0382
Square	5000	18	0.2180	2.7960	0.1819
Square	10000	25	0.4700	7.6710	0.3305
Square	50000	30	2.6197	45.9220	2.1597

Table 2: Benchmark results for the three convex hull algorithms using the EPICK kernel.

(d) By comparing the EPICK kernel with Simple Cartesian, it is easy to observe that convex hull computation takes longer, with a huge difference to be observed for Jarvis in the case of the circle with 50000 points on the hull. Using EPICK, computation of the orientation becomes more expensive since it is done with exact precision opposed to the Simple Cartesian kernel. For Jarvis where the points are not sorted and the total amount of CCW to be done is $O(n^2)$ opposed to the other two, the difference is even more noticeable.

(b)

Kernel	Points	Hull size	Graham–Andrew (ms)	Jarvis (ms)	Akl–Toussaint (ms)
EPICK	50003	3	2.6204	4.4994	0.3609
Simple Cartesian	50003	3	2.1870	0.2379	0.2341

Table 3: Benchmark results for the triangular pointset with 50,000 interior points.

(b) & (d) & (c) A triangular point set was generated with 50,000 interior points and only 3 points on the convex hull. Therefore, the hull size is $h = 3$. This input is theoretically favorable for Jarvis March. Since Jarvis March has running time $O(nh)$ and in this case the hull is only 3, this comes in agreement with the experimental results acquired by `Simple_cartesian<double>` kernel, where Jarvis March is much faster than Graham–Andrew.

However, with the EPICK kernel, Jarvis March is slower than Graham–Andrew since the cost of each geometric predicate increases and Jarvis does more orientation tests than the other two convex hull algorithms as shown above.

Jarvis March performs approximately $3 \cdot 50003$

Graham–Andrew also performs orientation tests, but only n of them after sorting.

Its main cost is the $O(n \log n)$ sorting step, which uses coordinate comparisons. Therefore, although Graham–Andrew has worse asymptotic complexity for this specific case, it is less sensitive to the exact-predicate cost of EPICK than Jarvis March.

Akl–Toussaint is the fastest algorithm for the triangular point set. This happens partially for the same reason as described before for this algorithm, that it grows faster the more the points from the pointset are internal.

What is interesting here, is how much better than Jarvis it performs even though they conduct the same number of orientation tests for this particular pointset. This might be due to the fact that Akl’s orientation tests are actually cheaper than Jarvis, since they are conducted with two fixed points (vertices of an edge of extreme points) and a point, which could be easier computed in most cases in contrast with Jarvis, whom to find a better convex hull candidate each time, the candidate and sample point could get closer to each other with each new sample point used causing EPICK to switch to more precise computations and thus becoming more expensive in general.

*EPICK uses filtered predicates. For an orientation test, it first tries to determine the sign using fast floating-point arithmetic. If the sign is clearly positive or negative, the result is returned immediately. If the value is too close to zero and rounding errors could change the sign, the predicate falls back to exact arithmetic. Therefore, EPICK is usually close to the speed of floating-point kernels, but remains robust in difficult near-degenerate cases.